
SAS BASE EXAM (A00-231)

Lecture 1

Easy Career

Chang Liu

Contents

About Exam	3
About SAS and SQL	4
Install SAS	5
Task 1: Boston House Data Analysis	5
Task 2: Exercise	8
Task 3: Data manipulation	11
Task 4: Exercise	13
Lab 1	14

About Exam

website:

https://www.sas.com/en_us/certification/credentials/foundation-tools/base-programming-specialist.html

- 20-25% - Access and create data structures.
- 35-40% - Manage data.
- 15-20% - Error handling.
- 15-20% - Generate reports and output.

Exam Detail:

- This exam is administered by SAS and Pearson VUE.
- 40-45 multiple choice and short-answer questions.
- 135 minutes to complete exam.
- Passing score is 725 (score range from 200 to 1,000 points).
- This exam is based on SAS 9.4 M5.

About SAS and SQL

SAS (Statistical analysis system) is one of the most popular software for data analysis. It is widely used for various purposes such as data management, data mining, report writing, statistical analysis, business modeling, applications development and data warehousing. Knowing SAS is an asset in many job markets. It is tagged “leader” in Advanced Analytics Platforms as per Gartner 2019 Magic Quadrant for Data Science & Machine Learning Platforms.

Why SAS is popular in job market?

SAS has over 40000 customers worldwide and holds largest market share in advanced analytics. It has been tagged “leader” consistently for last 6 consecutive years in advanced analytics platforms. In finance (BFSI) industry, SAS retains No. 1 spot and is being used as a primary tool for data manipulation and predictive modeling.

- Data Security - Because of unparalleled data security provided by SAS software, it is leading the analytics software industry in BFSI sector.
- Tech Customer Support - SAS provides one of the best tech support. If you are stuck in anything related to SAS whether it is installation related issue or clarity in any of SAS functions and procedures, they have both online and offline community to support you.
- Detailed Documentation - SAS documentation is very detailed as compared to open source software like R and Python.
- Memory Management - SAS can store datasets on hard drive and process bigger data set than size of your RAM.
- Stable Software more important than cost of software license : All the functions and procedures of previous software version are supported in new SAS versions. Cost of software license is a peanut to a bank or pharmaceutical company.
- Legacy System - Many banks have been using SAS for last 20-30 years and they have automated the whole process of analysis and have written millions of lines of working code. To convert all the stable reporting system from SAS to R/Python, it may require significant additional cost.

Install SAS

SAS University Edition

- Installation Required.
- Run on local machine (laptop / Desktop).
- No Internet Required,

SAS OnDemand for Academics

- No Installation Required. Set up your account by registering yourself.
- You can access it from anywhere as it is run on cloud.
- Internet Required.
- Don't go by software name "Academics". It is available for everyone not just college students.

Task 1: Boston House Data Analysis

1. set the libname

```
/* Specify the libname */  
libname bh '/folders/myfolders/Boston House';
```

2. Read the Boston House Pricing data into SAS and name it boston_house

```
/* Specify the libname */  
libname bh '/folders/myfolders/Boston House';  
proc import datafile='/folders/myfolders/Boston House/boston_house_train.csv'  
  dbms = csv  
  out= bh.boston_house  
  replace;  
  getnames= yes;  
run;  
proc export data=bh.boston_house dbms=xlsx  
  outfile = '/folders/myfolders/Boston House/boston_house_train.xlsx';  
run;  
proc export data=bh.boston_house dbms=tab
```

```
outfile = '/folders/myfolders/Boston House/boston_house_train.txt';  
run;
```

3. Read through the data dictionary to make sure that you are familiar with the data. Now, drop the price variable to create a new dataset with the name boston_house_features.

```
data bh.boston_house_features;  
set bh.boston_house;  
drop medv;  
run;
```

4. Create another new dataset with ID and price only, name it boston_house_price.

```
data bh.boston_house_price;  
set bh.boston_house;  
keep ID medv;  
run;
```

5. Create a dataset (name boston_house_charles) which only contains houses that are around the Charles river.

```
data bh.boston_house_charles;  
set bh.boston_house;  
where chas = 1;  
run;  
data bh.boston_house_charles;  
set bh.boston_house;  
if chas = 1;  
run;
```

6. Create a dataset (name boston_house_age_ind) which contains a new variable age_ind that distinguishes whether the proportion of units build prior to 1940 is higher than 30% or not.

```
data bh.boston_house_age_ind;  
set bh.boston_house;  
if age >= 30 then age_ind = 1;  
else age_ind = 0;  
run;
```

7. Read through the following SAS script and find if there is a problem with the logic.

```
data bh.boston_house_age_ind_2;  
set bh.boston_house;  
if age > 100 then age_ind = 1;  
else if age < 100 then age_ind = 0;  
run;
```

8. Now run the script from question 6. Create a new dataset (name boston_house_null_ind) which contains all of the data with missing age indicator.

```
data bh.boston_house_null_ind;  
set bh.boston_house_age_ind_2;  
where age_ind = .;  
run;  
  
data bh.boston_house_null_ind;  
set bh.boston_house_age_ind_2;  
where missing(age_ind);  
run;
```

Task 2: Exercise

1. Read the Boston House Pricing test data into SAS and name it boston_house_test

```
proc import datafile='/folders/myfolders/Boston House/boston_house_test.csv'
dbms = csv
out= bh.boston_house_test
replace;
getnames= yes;
run;
proc contents data = bh.boston_house_test;
run;
proc export data=bh.boston_house_test dbms=xlsx
outfile = '/folders/myfolders/Boston House/boston_house_test.xlsx';
run;
proc export data=bh.boston_house_test dbms=tab
outfile = '/folders/myfolders/Boston House/boston_house_test.txt';
run;
```

2. Read through the data dictionary to make sure that you are familiar with the data. Now, create a dataset which only contains the information regarding the physical characteristics of the houses.

```
data bh.boston_house_physical;
set bh.boston_house_test;
keep ID rm age;
run;
```

3. If the analysis does not want to include variables related to education and future employment opportunity, how would you modify the dataset? Create a new dataset (boston_house_test_remove_eduemp) for this.

```
data bh.boston_house_test_remove;
set bh.boston_house_test;
drop indus dis ptratio;
run;
```

4. Create a dataset (name boston_house_test_safe) which only contains records with per capita crime rate lower than 1%


```
data bh.boston_house_test_safe;
set bh.boston_house_test;
where crim < 1;
run;
data bh.boston_house_test_safe;
set bh.boston_house_test;
if crim < 1;
run;
```

5. For a town, if the average number of rooms per dwelling is higher than 6 and the proportion of old owner-occupied units is lower than 80%), then we say that the region's fancy_house indicator is "Y".otherwise it would be 'N'. Create a new variable in boston_house_test to capture that.

```
data bh.boston_house_test;
set bh.boston_house_test;
if rm > 6 and age < 80 then fancy_house = 'Y';
else fancy_house = 'N';
run;
```

6. If the pupil-teacher ratio is lower than 20 then we say the edu_ind = "good" else it would be "ok". Create a new variable in boston_house_test to capture that.

```
data bh.boston_house_test;
set bh.boston_house_test;
if ptratio < 20 then edu_ind = 'good';
else edu_ind = 'ok';
run;
```

7. If both of the fancy house and education indicator are positive, then we say that the house has quality_ind as Super, if the fancy house is positive but the education indicator is negative, then we say that the quality_ind is "Nice to live", if the education indicator is positive but the fancy house is negative, then we say that the quality_ind is "Good education", if either of them is positive, then the quality_ind is "No good". Create a new variable for that in boston_house_test. Note: you may find some issue with the actual content of this variable. This is due to the variable length. Do not worry, we will cover it when we take up the homework.* /

```
data bh.boston_house_test_v2;
set bh.boston_house_test;
length quality_ind_2 $20;
if fancy_house = 'Y' and edu_ind = 'good' then quality_ind_2 = 'Super';
else if fancy_house = 'Y' and edu_ind = 'ok' then quality_ind_2 = 'Nice to live';
else if fancy_house = 'N' and edu_ind = 'good' then quality_ind_2 = 'Good education';
else quality_ind_2 = 'No good';
run;
```

Task 3: Data manipulation

1. Use proc contents to understand the structure of the data.

```
proc contents data=bh.boston_house_test_v2;  
run;
```

2. Use proc freq to find the distribution of the quality_ind.

```
proc freq data=bh.boston_house_test_v2;  
table quality_ind_2 /nocum;  
run;
```

3. Use proc freq to find the distribution of data by chas and quality_ind.

```
proc freq data=bh.boston_house_test_v2;  
table chas*quality_ind_2/list;  
run;
```

4. Use proc means to understand the statistical properties of the price (medv).

```
proc means data = bh.boston_house_train;  
var medv;  
run;
```

5. Use the same logic of the last homework to create fancy_house, edu_ind, and quality_ind for the boston_house dataset. Check the how does the price (medv) change by the quality_ind.

```
data bh.boston_house_train;  
set bh.boston_house_train;  
if rm > 6 and age < 80 then fancy_house = 'Y';  
else fancy_house = 'N';  
if ptratio < 20 then edu_ind = 'good';  
else edu_ind = 'ok';  
run;  
data bh.boston_house_train;  
set bh.boston_house_train;  
length quality_ind $20;  
if fancy_house = 'Y' and edu_ind = 'good' then quality_ind = 'Super';
```

```
else if fancy_house = 'Y' and edu_ind = 'ok' then quality_ind = 'Nice to live';
else if fancy_house = 'N' and edu_ind = 'good' then quality_ind = 'Good education';
else quality_ind = 'No good';
run;
proc means data = bh.boston_house_train;
class quality_ind;
var medv;
run;
```

6. Use proc univariate to explore the statistical properties of the price (medv) and generate a histogram for it.

```
proc univariate data = bh.boston_house_train;
var medv;
histogram;
run;
```

7. Use proc univariate to explore the statistical properties of the price (medv) and generate a histogram for data with quality_ind = “Super” only.

```
proc univariate data = bh.boston_house;
where quality_ind='Super';
var medv;
histogram;
run;
```

8. Use proc univariate to explore the statistical properties of the price (medv) by quality_ind. Generate four histograms and compare the difference.

```
proc univariate data = bh.boston_house;
class quality_ind;
var medv;
histogram;
run;
```

Task 4: Exercise

1. Use proc contents to get familiar with the structure of the data.

```
proc contents data=email.email_data;  
run;
```

2. Check the distribution of the data by email type (Segment)

```
proc freq data=email.email_data;  
table segment;  
run;
```

3. Find the statistical properties of visit, conversion, and spend.

```
proc means data= email.email_data;  
var visit conversion spend;  
run;
```

4. Find the statistical properties of visit, conversion, and spend by email type (Segment).
Which e-mail campaign performed the best, the Mens version, or the Womens version?

```
proc means data= email.email_data;  
class segment;  
var visit conversion spend;  
run;
```

Lab 1

Example 1

```
DATA temp1;
  input subj 1-4 gender 6 height 8-9 weight 11-13;
  DATALINES;
1024 1 65 125
1167 1 68 140
1168 2 68 190
1201 2 72 190
1302 1 63 115
;
RUN;

PROC PRINT data=temp1;
  title 'Output dataset: TEMP1';
RUN;
```

Example 2 “r libname ex “/folders/myfolders/base exercise”; *Specifies the SAS data library (directory);

```
DATA ex.temp2; input subj 1-4 gender 6 height 8-9 weight 11-13; DATALINES; 1024 1 65 125
1167 1 68 140 1168 2 68 190 1201 2 72 190 1302 1 63 115 ; RUN;
```

```
PROC PRINT data=ex.temp2; title “Output dataset: STAT480.TEMP2”; RUN; ““
```

Example 3

```
DATA temp3;
  infile '/folders/myfolders/base exercise/temp3.dat';
  input subj 1-4 gender 6 height 8-9 weight 11-13;
RUN;

PROC PRINT data=temp3;
  title 'Output dataset: TEMP3';
RUN;
```

Example 4

```
DATA temp;
  input init $ 6 f_name $ 6-16 l_name $ 18-23
         weight 30-32 height 27-28;
  CARDS;
1024 Alice      Smith  1 65 125
1167 Maryann    White  1 68 140
1168 Thomas     Jones  2   190
1201 Benedictine Arnold 2 68 190
1302 Felicia    Ho     1 63 115
;
RUN;

PROC PRINT data=temp;
  title 'Output dataset: TEMP';
RUN;
```

Example 5

```
DATA grades;
  input name $ 1-15 e1 e2 e3 e4 p1 f1;
  * add up each students four exam scores
  and store it in examtotal;
  examtotal = e1 + e2 + e3 + e4;
  DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon       88 72 86  . 100 85
Patricia Jones   98 92 92 99  99 93
Jack Benedict    54 63 71 49  82 69
Rene Porter      100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
  var name e1 e2 e3 e4 examtotal;
RUN;
```

Example 6

```
DATA grades;
  input name $ 1-15 e1 e2 e3 e4 p1 f1;
  e2 = e2 + 8; * add 8 to each student's
               second exam score (e2);

  DATALINES;
Alexander Smith 78 82 86 69 97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99 99 93
Jack Benedict   54 63 71 49 82 69
Rene Porter     100 62 88 74 98 92
;
RUN;

PROC PRINT data = grades;
  var name e1 e2 e3 e4 p1 f1;
RUN;
```

Example 7

```
DATA grades;
  input name $ 1-15 e1 e2 e3 e4 p1 f1;
  final = 0.6*((e1+e2+e3+e4)/4) + 0.2*p1 + 0.2*f1;
  DATALINES;
Alexander Smith 78 82 86 69 97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99 99 93
Jack Benedict   54 63 71 49 82 69
Rene Porter     100 62 88 74 98 92
;
RUN;

PROC PRINT data = grades;
  var name e1 e2 e3 e4 p1 f1 final;
RUN;
```

Example 8

```
DATA grades;
  input name $ 1-15 e1 e2 e3 e4 p1 f1;
  * calculate the average by definition;
  avg1 = (e1+e2+e3+e4)/4;
```



```
    * calculate the average using the mean function;
    avg2 = mean(e1,e2,e3,e4);
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99  99 93
Jack Benedict   54 63 71 49  82 69
Rene Porter     100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name e1 e2 e3 e4 avg1 avg2;
RUN;
```

Example 9

“r DATA grades; input name \$ 1-15 e1 e2 e3 e4 p1 f1; *calculate the average using the mean function and then round it to the nearest digit; avg = round(mean(e1,e2,e3,e4),1); DATALINES; Alexander Smith 78 82 86 69 97 80 John Simon 88 72 86 . 100 85 Patricia Jones 98 92 92 99 99 93 Jack Benedict 54 63 71 49 82 69 Rene Porter 100 62 88 74 98 92 ; RUN;

PROC PRINT data = grades; var name e1 e2 e3 e4 avg; RUN; “r

Example 10

```
DATA grades;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    * calculate the average using the mean function;
    avg = mean(e1,e2,e3,e4);
    * if the average is less than 65 indicate failed,
      otherwise indicate passed;
    if (avg < 65) then status = 'Failed';
    else status = 'Passed';
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99  99 93
Jack Benedict   54 63 71 49  82 69
Rene Porter     100 62 88 74  98 92
;
RUN;
```

```
PROC PRINT data = grades;
    var name e1 e2 e3 e4 avg status;
RUN;
```

Example 11

```
DATA grades;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    * if the first exam is less than 65 indicate failed;
    if (e1 < 65) then status = 'Failed';
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon       88 72 86  . 100 85
Patricia Jones   98 92 92 99  99 93
Jack Benedict    54 63 71 49  82 69
Rene Porter      100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name e1 status;
RUN;
```

Example 12

```
DATA grades;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    if p1 in (98, 99, 100) then project = 'Excellent';
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon       88 72 86  . 100 85
Patricia Jones   98 92 92 99  99 93
Jack Benedict    54 63 71 49  82 69
Rene Porter      100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name p1 project;
RUN;
```

Example 13

```
DATA grades;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    * if the first exam is less than 65 indicate failed;
    if (e1 < 65) then status = 'Failed';
    * otherwise indicate passed;
    else status = 'Passed';
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99  99 93
Jack Benedict   54 63 71 49  82 69
Rene Porter     100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name e1 status;
RUN;
```

Example 14

```
DATA grades;
    length status $ 6;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    * if the fourth exam is missing indicate missing;
    * else if the fourth exam is less than 65 indicate failed;
    * otherwise indicate passed;
    if (e4 = .) then status = ' ';
    else if (e4 < 65) then status = 'Failed';
    else status = 'Passed';
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99  99 93
Jack Benedict   54 63 71 49  82 69
Rene Porter     100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
```

```
var name e4 status;  
RUN;
```

Example 15

```
DATA grades;  
  length overall $ 10;  
  input name $ 1-15 e1 e2 e3 e4 p1 f1;  
  avg = round((e1+e2+e3+e4)/4,0.1);  
    if (avg = .) then overall = 'Incomplete';  
  else if (avg >= 90) then overall = 'A';  
  else if (avg >= 80) and (avg < 90) then overall = 'B';  
  else if (avg >= 70) and (avg < 80) then overall = 'C';  
  else if (avg >= 65) and (avg < 70) then overall = 'D';  
  else if (avg < 65) then overall = 'F';  
  DATALINES;  
Alexander Smith 78 82 86 69 97 80  
John Simon      88 72 86 . 100 85  
Patricia Jones  98 92 92 99 99 93  
Jack Benedict   54 63 71 49 82 69  
Rene Porter     100 62 88 74 98 92  
;  
RUN;  
  
PROC PRINT data = grades;  
  var name avg overall;  
RUN;
```

Example 16

```
DATA grades;  
  length overall $ 10;  
  input name $ 1-15 e1 e2 e3 e4 p1 f1;  
  avg = round((e1+e2+e3+e4)/4,0.1);  
    if (avg EQ .) then overall = 'Incomplete';  
  else if (90 LE avg LE 100) then overall = 'A';  
  else if (80 LE avg LT 90) then overall = 'B';  
  else if (70 LE avg LT 80) then overall = 'C';  
  else if (65 LE avg LT 70) then overall = 'D';  
  else if (0 LE avg LT 65) then overall = 'F';  
  DATALINES;
```

```
Alexander Smith  78 82 86 69  97 80
John Simon       88 72 86  . 100 85
Patricia Jones   98 92 92 99  99 93
Jack Benedict    54 63 71 49  82 69
Rene Porter      100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name avg overall;
RUN;
```

Example 17

```
DATA grades;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    avg = round((e1+e2+e3+e4)/4,0.1);
    if    ((e1 < e3) and (e1 < e4))
        or ((e2 < e3) and (e2 < e4)) then adjavg = avg + 2;
    else adjavg = avg;
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon       88 72 86  . 100 85
Patricia Jones   98 92 92 99  99 93
Jack Benedict    54 63 71 49  82 69
Rene Porter      100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name e1 e2 e3 e4 avg adjavg;
RUN;
```

Example 18

```
DATA grades;
    length action $ 7
          action2 $ 7;
    input name $ 1-15 e1 e2 e3 e4 p1 f1 status $;
    if (status = 'passed') then action = 'none';
    else if (status = 'failed') then action = 'contact';
```

```
    else if (status = 'incomp') then action = 'contact';
        if (upcase(status) = 'PASSED') then action2 = 'none';
    else if (upcase(status) = 'FAILED') then action2 = 'contact';
    else if (upcase(status) = 'INCOMP') then action2 = 'contact';
    DATALINES;
Alexander Smith  78 82 86 69  97 80 passed
John Simon      88 72 86  . 100 85 incomp
Patricia Jones  98 92 92 99  99 93 Passed
Jack Benedict   54 63 71 49  82 69 FAILED
Rene Porter     100 62 88 74  98 92 PASSED
;
RUN;

PROC PRINT data = grades;
    var name status action action2;
RUN;
```

Example 19

```
DATA grades;
    input name $ 1-15 e1 e2 e3 e4 p1 f1;
    if e4 = . then do;
        e4 = 0;
        notify = 'YES';
    end;
    DATALINES;
Alexander Smith  78 82 86 69  97 80
John Simon      88 72 86  . 100 85
Patricia Jones  98 92 92 99  99 93
Jack Benedict   54 63 71 49  82 69
Rene Porter     100 62 88 74  98 92
;
RUN;

PROC PRINT data = grades;
    var name e1 e2 e3 e4 p1 f1 notify;
RUN;
```